

CI/CD Portfolio Deployment

◆ 1. Objective

To automatically deploy a portfolio website to an EC2 instance using GitHub Actions.

👉 **CI/CD Meaning:**

- **CI (Continuous Integration)** → Code push → build & test
- **CD (Continuous Deployment)** → Code automatically deploy

👉 Simple:

No manual work → Everything automatic

Step 1: Go to Project Folder

Cd portfolio

👉 Go to your project folder

Step 2: Create CI/CD Folder

Create a folder for cicd pipeline — **.github/workflows**

- GitHub Actions reads files from this folder
- This is a **standard folder structure**

Step 3: Create Pipeline File

Create a file → **portfolio.yml**

- **.yml** file = configuration file
 - It tells GitHub:
 - When to run
 - What steps to execute
-

Step 4: Add Secrets

Go to repository → **portfolio**

Go to **setting**

Click **secrets and variables**

Click **actions**

Click **new repository secret**

Paste **secret value**

Create **secrets**

- Secrets are **secure environment variables**
- Used to store sensitive data (keys, passwords)

Add these secrets:

- **AWS_ACCESS_KEY_ID** → AWS login key
- **AWS_SECRET_ACCESS_KEY** → AWS secret key
- **AWS_REGION** → AWS region
- **EC2_INSTANCE_ID** → EC2 ID
- **EC2_SSH_KEY** → SSH private key

👉 Security:

- Secrets are hidden (not visible in code)
-

Step 5: Paste CI/CD Code in **portfolio.yml**

```
name: Deploy Portfolio to EC2
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout Code
```

```
        uses: actions/checkout@v3
```

```
      - uses: aws-actions/configure-aws-credentials@v2
```

```
        with:
```

```
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
```

```
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

```
          aws-region: ${ secrets.AWS_REGION }
```

```
      - name: Fetch EC2 public IP
```

```
        id: getip
```

```
run: |

    INSTANCE_ID=${{ secrets.EC2_INSTANCE_ID }}

    REGION=${{ secrets.AWS_REGION }}

    IP=$(aws ec2 describe-instances --instance-ids "$INSTANCE_ID" --region "$REGION"
--query "Reservations[0].Instances[0].PublicIpAddress" --output text)

    echo "ip=$IP" >> $GITHUB_OUTPUT

- name: Deploy via SSH

  uses: appleboy/ssh-action@v1.0.0

  with:

    host: ${ steps.getip.outputs.ip }

    username: ubuntu

    key: ${ secrets.EC2_SSH_KEY }

  script: |

    cd ~/portfolio

    git pull origin main

    npm install

    npm run build

    pm2 restart portfolio-dev || pm2 start npm --name "portfolio-dev" -- start
```

Theory of Pipeline Steps

✓ on: push

👉 Trigger condition

- When code is pushed → pipeline starts
-

✓ runs-on: ubuntu-latest

👉 GitHub gives a temporary server

✓ checkout

👉 Downloads your code to runner

✓ AWS Credentials

👉 Login to AWS using secrets

✓ Fetch EC2 IP

👉 Uses AWS CLI to get public IP dynamically

✓ SSH Deployment

👉 Connects to EC2 and runs commands

✓ PM2

- 👉 Keeps your Node app running
 - 👉 Restart app after deployment
-

Step 6: Update Portfolio Code (Local)

Update portfolio website code on the local

- You make changes in code
 - Then push to GitHub
-

Step 7: Run Commands

`git status`

- 👉 Check file changes

`git add .`

- 👉 Add files

`git commit -m "testing cicd pipeline"`

- 👉 Save changes

`git push origin main`

- 👉 Push to GitHub
-

Step 8: Automatic CI/CD Run

- Push triggers GitHub Actions

- Pipeline executes all steps automatically

Step 9: Final Result

Update website on the EC2 instance

👉 Website automatically updated

Full Flow

1. Developer writes code
2. Push to GitHub
3. CI/CD pipeline starts
4. AWS login
5. EC2 connection
6. Code update + build
7. App restart

Final Simple Line

👉 Push code → Auto deploy → Website updated 🚀